Chapter 13
Compiler Construction

Abstract A compiler refers to a software tool or system which performs automatic conversion from one computer language to another. Along the way, the compiler tries to optimize the program while maintaining the semantics of the computation. This chapter discusses the compiler construction tools `flex`, `bison`, and LLVM. Related text processing tools such as `m4`, `gperf` and `readline` are also presented. We also discuss the various GNU binutils tools, including `ar`, `nm` and `ld`. Examples using the various compiler construction tools are presented.

Contents

13.1 Introduction **241**
13.2 Anatomy of a Compiler **242**
13.3 Lexical Analysis **243**
13.4 YACC: Yet Another Compiler Compiler **250**
13.5 Code Generation **256**
13.6 LLVM: Low Level Virtual Machine **273**
13.7 Conclusion **284**

13.1 Introduction

In this chapter we discuss language processors and compiler construction tools. In Section 13.2 we introduce the compiler construction concept. The flowchart for language processing and translation is presented, and appropriate software tools for each part of the flow are introduced. In Section 13.3 we discuss lexical analysis. This problem has elegant mathematical roots and solutions from finite automata are discussed. Utilities such as `m4`, `readline`, `getopt` are also introduced. In Section 13.4 venerable parsing tool YACC and the ANTLR compiler construction toolkit are described. The Boost SPIRIT framework is also described in 13.4.1.

In Section 13.5 we describe the code generation, and instruction representation libraries available through GNU binutils. Other system tools such as linkers, archive managers, ELF inspection are also discussed.

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_13, © Springer Science+Business Media, LLC 2011

The Low Level Virtual Machine (LLVM) intermediate representation, and compiler optimization system is discussed in Section 13.6. GCC plugin `dragonegg`, the LLVM bit-code analyzer, and compiler optimization passes are discussed.

13.2 Anatomy of a Compiler

A compiler refers to a software tool or system which performs automatic conversion from one computer language to another. Along the way, the compiler tries to optimize the program while maintaining the semantics of the computation. The history of compiler construction tools is rich and varied. Classical texts such as Aho, Ullman and Sethi's Dragon Book have defined the field. As shown in Figure 13.1 the process of compiling a source text can be broken into (i) lexical analysis, (ii) parsing, (iii) intermediate code representation, (iv) optimization (which includes register allocation, scheduling and rewriting), and (v) finally, instruction generation.

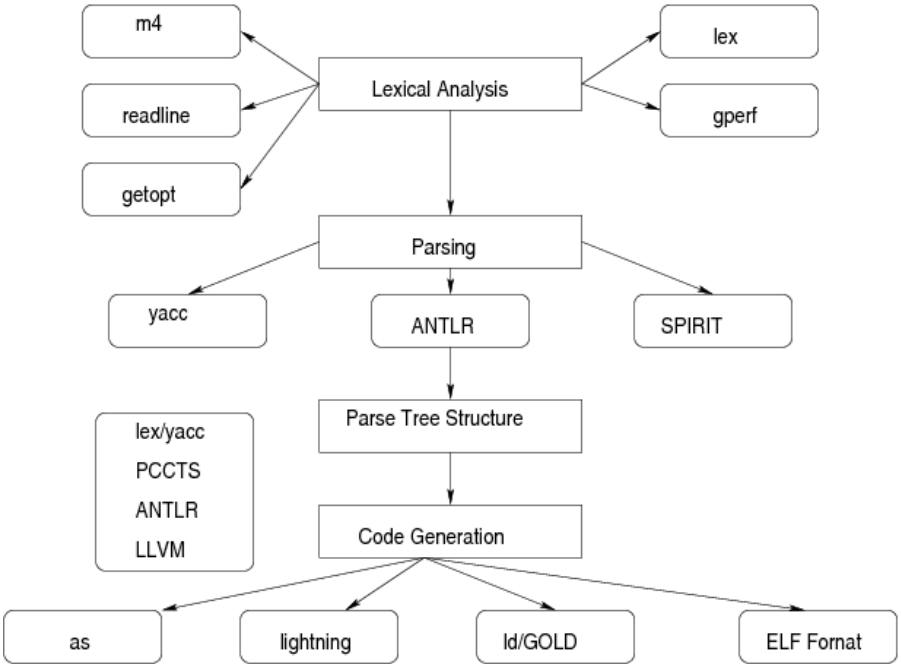


Fig. 13.1 Flow chart of compiler construction

Alongwith the compiler proper, the act of compiling also includes ancillary activities such as (i) assembler, (ii) loaders and linkers, (iii) binary object file manipulation, and (iv) library archive maintenance. Towards this end, in this part we have also discussed the important and ubiquitous GNU `bin-utils`. More details are given below in Section 13.5.1 Intermediate representation (IR) and compiler optimizations are discussed later in Section 13.6.

13.3 Lexical Analysis

Lexical analysis is defined as the task of *tokenizing* a given input stream according to pre-defined rules. Examples of lexical analysis abound in computer literature, from command line parsing, macro substitution, UNIX `grep`, to actual programming language compilers. The problem can be modeled as a halting problem on finite automata. The rules for lexical analysis are input by the user, and most commonly a software tool is used to construct a lexical scanner. The software usually also optimizes the scanner so that it reduces memory usage during scanning, and optimizes the performance. The common lexical analysis generator is `lex`, and its GNU implementation is called `flex`. This is discussed below in Subsection 13.3.1.

As we mentioned above, lexical analysis is also used in `grep` (see Chapter 1.3), macro processor (`m4`), GNU `readline`, and the GNU perfect hash generator (`gperf`). These are discussed below.

13.3.1 GNU flex

GNU `flex` automatically generates programs whose control flow is directed by instances of regular expressions in the input stream. It is well suited for generating lexical analyzers and token processors, and for segmenting input in preparation for a parsing routine.

GNU `flex` source is a table of regular expressions (called *rules*) and corresponding program fragments, called *actions*. The table is translated to a program by `flex` which reads an input stream, copying it to an output stream and partitioning the input into strings which match the given expressions. Each such string is recognized using deterministic finite automata (DFA); when the string is matched the corresponding program fragment is executed. GNU `flex` generates as output a C source file named 'lex.yy.c' (by default), which defines a routine `yylex()`. This file is compiled and linked with the '-lfl' library to produce an executable. The following flex input specifies a scanner which whenever it encounters the string "username" will replace it with the user's login name:

```
%%
username    printf( "%s", getlogin() );
```

By default, any text not matched by a flex scanner is copied to the output, so the net effect of this scanner is to copy its input file to its output with each occurrence of "username" expanded.

13.3.1.1 Format of the input file

The GNU `flex` input file consists of three subsections, separated by a line with just %% in it:

```
definitions
%%
rules
%%
user code
```

The definitions subsection contains declarations of simple name definitions to simplify the scanner specification, and declarations of start conditions. The command-line usage of GNU `flex` is:

```
Usage: flex [OPTIONS] [FILE]...
Generates programs that perform pattern-matching on text.

Debugging:
  -p, --perf-report write performance report
  -v, --verbose     write summary of scanner statistics

Files:
  -o, --outfile=FILE      specify output filename
  -S, --skel=FILE         specify skeleton file
  -t, --stdout            write scanner on stdout
      --yyclass=NAME      name of C++ class
      --header-file=FILE   create a C header file
      --tables-file[=FILE] write tables to FILE
  -i, --case-insensitive  ignore case in patterns
      --yylineno          track line count in yylineno

  -+,  --c++               generate C++ scanner class
  -Dmacro[=defn]
       #define macro defn  (default defn is '1')
```

A more complete list of command line options can be found by running `man flex`. Now we present some example of using GNU `flex` to generate a scanner.

```
/* A simple lexical scanner
   (C) Sandeep Koranne, 2010.
*/
%{
#include <stdio.h>
#include <unistd.h>
  int num_lines = 0;
  char login_name[1024];
%}

%%
\n   { ++num_lines; printf("\n"); }
username { cuserid( login_name ); printf( "%s", login_name ); }
%%
int main()
{
  yylex();
  printf( "# of lines = %d\n", num_lines );
}
```

Listing 13.1 Lex exampe file for username analysis

We generate a scanner using GNU `flex` as:

```
$flex -o username.c username.l
$gcc -o substitute_username username.c -lfl
$ cat scanner_example.txt
This is a simple report generator.
username is writing this.
$ cat scanner_example.txt | ./substitute_username
This is a simple report generator.
skoranne is writing this.
# of lines = 2
```

A slightly more involved example (coming from a Pascal compiler) is shown below:

```
";" { return SEMI; }
":" { return COLON;}
"," { return COMA; }
"("     { return LP; }
")"     { return RP ; }
"[" { return LB ; }
"]" { return RB ; }
":="    { return ASSGN ; }
and { return AND_TOK ; }
array { return ARRAY;  }
[bB][eE][gG][iI][nN]{ return BEGIN_TOK; }
case { return CASE; }
div { return DIV_TOK; }
do { return DO; }
downto { return DOWNTO; }
else { return ELSE; }
end { return END; }
for { return FOR; }
function{ return FUNCTION ; }
if { return IF ; }
```

This conversion of fixed size and constant strings (mostly coming from the reserved words and keywords of programming languages) into tokens is so common that a special tool GNU `gperf` has been developed which performs *perfect hashing*. This makes the lexical scanner perform much faster at this task of identifying keywords.

13.3.1.2 GNU gperf: the perfect hash generator

GNU `gperf` reads in a set of keywords from a user-provided file (which typically has a '.gperf' extension, although this is not mandatory)for example, commandoptions.gperfand generates C/C++ sources for the hash table, hashing, and lookup methods. All code is written to standard output (by default).

The input file for `gperf` is similar in format to GNU `flex`; it contains

```
%{
/* C code */
%}
declarations
%%
keywords
%%
functions
```

The declarations subsection is optional and can be omitted as long as GNU `gperf` is not invoked with the -t option. In case the -t option is used, the last component in the declaration subsection must be a structure whose first field must be a char* or const char* identifier called name.

The next construct in the file is the *keywords* subsection. Each line in this subsection that begins with the number sign (#) in the first column is a comment. The keywords are the first field of each non-comment line in the keywords subsection. Using the Pascal example from above:

```
/*
 GNU gperf input file for the Pascal language
 (C) Sandeep Koranne, 2010
*/
%{
#include <stdio.h>
#include <string.h>
%}
%%
#Pascal keywords
and, AND_TOK
array, ARRAY
begin, BEGIN_TOK
case, CASE
div, DIV_TOK
do, DO
downto, DOWNTO
else, ELSE
end, END
for, FOR
function, FUNCTION
if, IF
of, OF
or, OR_TOK
procedure, PROCEDURE
program, PROGRAM
then, THEN
to, TO
var, VAR_TOK
while, WHILE
write, WRITE_TOK
integer, INT_TYPE
real, REAL_TYPE
read, READ_TOK
repeat, REPEAT
until, UNTIL
goto, GOTO
%%
```

Listing 13.2 Example `gperf` input file for Pascal keywords

A simple test harness to exercise the GNU `gperf` generated keyword checker compared to a string comparison function was also written.

```
/*
 Test harness for GNU gperf, performance characterization also
 (C) Sandeep Koranne, 2010
*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

extern const char *in_word_set (const char *str, unsigned int len );

static const char* PASCAL_KW[] = { "and", "array", "begin",
           "case", "div", "do", "downto",
          "else", "end", "for",
          "function", "if", "of", "or",
          "procedure", "program","then",
          "to", "var", "while", "write",
          "integer", "real", "read",
          "repeat", "until", "goto"
};

int main( int argc, char *argv[] )
{
  FILE *fp = NULL;
  int kw_found = 0;
  if( argc != 2 ) {
    fprintf( stderr, "usage: pascal_kw <filename>\n" );
    exit(1);
  }
  fp = fopen( argv[1], "rt" );
  if( !fp ) {
    fprintf( stderr, "Unable to open Pascal program file : %s", argv[1] );
    exit(1);
  }

  while( 1 ) {
    int i;
    char kw[1024];
    int  num_read = fscanf( fp, "%s", kw );
    if( num_read == 0 ) break;
    if( strcmp( kw, "end." ) == 0 ) { kw_found++; break; }
    #ifndef USE_GPERF
    for( i=0; i < 27;++i ) {
      if( strcmp( kw, PASCAL_KW[i] ) == 0 ) { kw_found++; break; }
    }
    #else
    if( in_word_set( kw, strlen( kw ) ) ) kw_found++;
    #endif
  }
  fclose( fp );
  printf("%d Pascal keywords found.\n", kw_found );
  return 0;
}
```

Listing 13.3 Test harness for `gperf` for Pascal keywords

On a collection of Pascal programs totalling 4.9 million lines of code containing 5.7 million Pascal keywords, the `gperf` based functions are almost 2 times faster than the string comparison function.

13.3.2 GNU m4

GNU `m4` is a macro processor, meaning that it copies its input to the output, expanding macros along the way. In this regard it's similar to the C pre-processor 'cpp'. Like cpp, `m4` originally was written as the pre-processor for a programming language (Rational FORTRAN); however, `m4` is much more powerful and feature-rich than 'cpp', which makes it much more useful than just defining constants in programs. Consider the simple example:

```
$cat expand_hello.m4
define(‘hello’, ‘Hello World from here’)
```

Note the particular use of quotation mark around the text to be macro processed. The format is intuitive, wherever the pattern exists, m4 will replace that with the defined macro. We run m4 as:

```
$cat m4_example.txt
Are you saying hello
$m4 expand_hello.m4 m4_example.txt
Are you saying Hello World from here
```

GNU `m4` writes the processed output on standard output. Some of the commonly used command line options for m4 are:

```
‘--help’
‘--version’
‘-E’
‘--fatal-warnings’
     Controls the effect of warnings.
‘-i’
‘--interactive’
‘-e’
     Makes this invocation of ‘m4’ interactive.

‘-P’
‘--prefix-builtins’

‘-Q’
‘--quiet’
```

GNU `m4` also allows for defining pre-processor macros on the command line itself.

```
‘-D NAME[=VALUE]’
‘--define=NAME[=VALUE]’

‘-I DIRECTORY’
‘--include=DIRECTORY’
```

```
'-U NAME'
'--undefine=NAME'
```

13.3.3 GNU readline

GNU `readline` reads a line from the terminal and returns it as a prompt. If prompt is NULL or the empty string, no prompt is issued. The line returned is allocated with `malloc`; the caller must free it when finished. The line returned has the final newline removed, so only the text of the line remains. GNU `readline` offers editing capabilities while the user is entering the line. By default, the line editing commands are similar to those of GNU `emacs`. Using GNU `readline` is simple as shown in the following API example:

```
#include <stdio.h>
#include <readline/readline.h>
#include <readline/history.h>
char * readline (const char *prompt);
```

13.3.4 getopt

The `getopt()` function parses the command-line arguments. Its arguments `argc` and `argv` are the argument count and array as passed to the `main()` function on program invocation. An element of `argv` that starts with '-' is an option element. The characters of this element (aside from the initial '-') are the option characters. If `getopt()` is called repeatedly, it returns successively each of the option characters from each of the option elements. The variable optind is the index of the next element to be processed in argv. If there are no more option characters, `getopt()` returns -1. Then optind is the index in argv of the first argv-element that is not an option. Consider the following API description:

```
  #include <unistd.h>
  int getopt(int argc, char * const argv[],
             const char *optstring);

5 extern char *optarg;
  extern int optind, opterr, optopt;
  #include <getopt.h>
```

The `getopt\_long()` function works like getopt() except that it also accepts long options, started with two dashes. A more detailed example from a real example is shown below:

```
int main( int argc, char* argv [] ) {
  int polymake_mode=0, graph_extraction_mode=0,latex_mode=0,reduced_mode=1;
  char *cano_file_name = NULL;
  int c;
5 dimension = 3; // by default
```

```
while (( c = getopt( argc, argv, "hprgGlf:d:m:") ) != -1 ) {
  switch( c ) {
  case 'f':
    cano_file_name = new char[ strlen(optarg) + 1];
    strcpy( cano_file_name, optarg );
    break;
  case 'd':
    dimension = atoi( optarg );
    break;
  case 'g':
    graph_extraction_mode = 1;
    break;
  case 'G':
    graph_extraction_mode = 2;
    break;
  case 'r':
    reduced_mode = 0;
    break;
  case 'l':
    latex_mode = 1;
    break;
  case 'm':
    SINGLE_SEPARATION_MODE = atoi( optarg );
    break;
  case 'p':
    polymake_mode = 1;
    break;
  case 'h':
  case '?':
    std::cerr << "pm_cd -f <cano> -d <dim> [ -p | -g ]\n";
    return 1;
  }
}
```

Listing 13.4 Example of using `getopt`

13.4 YACC: Yet Another Compiler Compiler

Yacc provides a general tool for describing the input to a computer program. The user specifies the structures of his input, together with code to be invoked as each such structure is recognized. Yacc turns such a specification into a subroutine that handles the input process. The GNU implementation of yacc is called `bison`. Bison is a general-purpose parser generator that converts a grammar description for an LALR(1) context-free grammar into a C (as well as C++) program to parse that grammar. Bison is upward compatible with Yacc. Input files should follow the yacc convention of ending in '.y'. Unlike yacc, the generated files do not have fixed names, but instead use the prefix of the input file. Usage:

```
Usage: bison [OPTION]... FILE
Generate LALR(1) and GLR parsers.
Operation modes:
  -h, --help                 display this help and exit
  -V, --version              output version information
  -y, --yacc                 emulate POSIX Yacc
```

```
Output:
  --defines[=FILE]            also produce a header file
  -d                          likewise
  -v, --verbose               same as `--report=state'
  -b, --file-prefix=PREFIX    specify a PREFIX for output files
  -o, --output=FILE           leave output to FILE
  -g, --graph[=FILE]          also output a graph of the automaton
```

A more comprehensive listing of command line options can be found by running `man bison`. An example of a simple library checkin, and checkout system can be constructed using GNU `flex` and GNU `bison`. The grammar of the library database control language is very simple; consider an example transcript shown below:

```
checkout book 98
checkout book 200
checkin book 76
checkin book 98
checkout book 23
checkout book 465
```

The grammar of the library database control language in BNF notation is the following:

```
commands: /* empty */
        | commands command
        ;
command:
        book_check_in | book_check_out
        ;
book_check_in: CHECKIN_TOKEN BOOK_TOKEN NUMBER_TOKEN
        { gTotalBooksCheckedOut--; }        ;
book_check_out: CHECKOUT_TOKEN BOOK_TOKEN NUMBER_TOKEN
        { gTotalBooksCheckedOut++; }        ;
```

The language consists of production rules which refer to tokens and other rules. To implement the lexical scanner which will translate the keywords `checkin`, `checkout` to a valid representation we use GNU `flex` with the following input file.

```
/*
 Simple library checkin, checkout controller.
 (C) Sandeep Koranne, 2010
*/
%{
#include <stdio.h>
#include "y.tab.h"
%}

%%
[0-9]+  return NUMBER_TOKEN;
book    return BOOK_TOKEN;
checkin return CHECKIN_TOKEN;
```

```
checkout return CHECKOUT_TOKEN;
\n      ;
[ \t]+ ;
%%
```

The GNU `bison` input file is shown in Listing 13.5.

```
/* \file lib.y
   \author Sandeep Koranne, (C) 2010
   \description Simple library checkin, checkout controller.

 Library Example:
 simple example is
 checkin book 76
 checkout book 80
*/
%{
#include <stdio.h>
#include <string.h>
  int gTotalBooksCheckedOut = 0;
  void yyerror( const char *str ) {
    fprintf( stderr, "syntax error: %s\n", str );
  }
  int yywrap() { return 1; }
  int main() {
    yyparse();
    printf("\n %d books still checked out.\n", gTotalBooksCheckedOut );
    return 0;
  }
%}

%token NUMBER_TOKEN BOOK_TOKEN CHECKIN_TOKEN CHECKOUT_TOKEN

%%
commands: /* empty */
        | commands command
        ;
command:
        book_check_in | book_check_out
        ;
book_check_in: CHECKIN_TOKEN BOOK_TOKEN NUMBER_TOKEN
        { gTotalBooksCheckedOut--; }        ;
book_check_out: CHECKOUT_TOKEN BOOK_TOKEN NUMBER_TOKEN
        { gTotalBooksCheckedOut++; }        ;
```

Listing 13.5 Example of GNU `bison` Yacc file

We generate the lexical scanner and parser using the above tools as:

```
$flex lib.l
$yacc -d lib.y
$gcc lex.yy.c y.tab.c -o library_system -lfl
$cat lib.txt | ./library_system
2 books still checked out.
```

At this time it is instructive to compare the EBNF grammar with the Yacc description we input to GNU `bison` (as shown in Listing 13.5). Mid-rule actions can also be specified in the rules sections; these can be used to call user defined function when part of a rule has been successfully matched.

13.4.1 Boost SPIRIT Framework

The Spirit Parser Framework is an object oriented recursive descent parser generator framework implemented using template meta-programming techniques. Expression templates allow users to approximate the syntax of Extended Backus Naur Form (EBNF) completely in C++. Parser objects are composed through operator overloading and the result is a backtracking LL(∞) parser that is capable of parsing rather ambiguous grammars. Spirit can be used for both lexing and parsing, together or separately. Consider the following example of parsing a file in the '.eqn' format:

```
a=b#c;
o=(c&d)$(e#!f);
```

The above '.eqn' file describes two equations, with output a as the boolean OR of b and c, and the output o as the Boolean XOR of two sub-expressions; c AND d and e OR NOT f. Such a format is common in VLSI CAD tools. The SPIRIT implementation of this parser is shown below:

```
struct EqnParser : public grammar<EqnParser> {
  template <typename ScannerT>
  struct definition  {
    definition(EqnParser const& /*self*/)  {
      expression
        = str_p("INPUT()") [&do_input]
        | str_p("OUTPUT()")[&do_output]
        | term
        >> *(    ('#' >> term)     [&do_or]
                 |   ('$' >> term)[&do_xor]
                 |   ('-' >> term)[&do_subt]
                 )
        ;

      term
        =   factor
        >> *(    ('&' >> factor)[&do_and]
                 |   ('/' >> factor)[&do_div]
                 )
        ;

      factor
        =   lexeme_d[(+digit_p)[&do_int]]
        |   lexeme_d[(+(alnum_p|'_'|'['|']'))[&do_literal]]
        |   '(' >> expression[&do_expr] >> ')'
        |   ('!' >> factor)[&do_neg]
        |   ('#' >> factor)
        ;

      assign
        =   lexeme_d[(+(alnum_p|'_'|'['|']'))[&do_lhs]]
        >> '=' >> expression[&do_final] >> ';'
        ;
    }
```

```
    rule<ScannerT> expression, term, factor, assign;
    rule<ScannerT> const&
    start() const { return assign; }
  };
};
```

The parser is modeled after a classic recursive descent parser for parsing mathematical expressions of the form:

```
start <- assignment;
assignment <- lhs '=' rhs;
lhs <- literal;
rhs <- expression;
expression <- '(' expression ')' | term op expression;
op <- XOR | NEG;
term <- term AND factor;
factor <- literal | literal OR factor;
```

Using Boost SPIRIT framework, the formal grammar is translated to C++ code using the following conventions: (i) production choices are delineated using the `|` operator, (ii) Kleene star closures are defined using the '*' operator, (iii) the stream redirection operator `>>` is overloaded to define production rules, and (iv) the array index operator '[]' is overloaded to define the function to be called when the production is matched. In the above example, we see that when the operator '#' is matched for example, the `do_or` function is called. The function to be called is defined to accept the *left* and *right* operand, i.e.:

```
void do_xor(char const* l, char const* r) {
  // take the 2 top elements from the stack
  // and make their tree and add to sub tree
  Tree* pT = new Tree( XOR, GetName() );
  pT->left = treeStack.top();   treeStack.pop();
  pT->right = treeStack.top();  treeStack.pop();
  treeStack.push(pT);
}
```

Consider a '.eqn' file comprising of a single equation:

```
o1 = !a # !b;
```

When we run our equation parser program with this input we get:

```
$./eqn < b.eqn
LHS = (o1)
PUSH LIT(a)
NEG
 PUSH LIT(b)
NEG
 ADD
Parsing succeeded

InSet  has 2 inputs
```

```
OutSet has 1 outputs
INORDER = a b  ;
OUTORDER = o1  ;

o1 = T2;
T2 = T1 + T0;
T1 = !b;
T0 = !a;
```

The parser automatically generates the NEG function calls to compute the complement of a to get a', similarly for b. These are then ORed together to produce $T2$ which is assigned to the output $o1$. The complete equation parser program includes logic to figure out the input set (any literal which is not assigned), and the output set (any literal which is assigned, but is itself not an operand in any expression). The final set of outputs is written out to standard out. A more complex example is shown below:

```
a=INPUT();
b=INPUT();
c=INPUT();
d=INPUT();
o1=OUTPUT();
o2=OUTPUT();
o1 = (!a # !b) & (!c $ (a&d) );
o2 = (!e $ f) $ ( a # !b);
o3 = (f $ e) & (a # b);
```

The generated equations are:

```
INORDER = a b c d e f  ;
OUTORDER = o1 o2 o3  ;

o1 = T6;
T6 = T5 * T2;
T5 = T4 * !T3 + ! T4 * T3 ;
T4 = d * a;
T3 = !c;
T2 = T1 + T0;
T1 = !b;
T0 = !a;

o2 = T11;
T11 = T10 * !T8 + ! T10 * T8 ;
T10 = T9 + a;
T9 = !b;
T8 = f * !T7 + ! f * T7 ;
T7 = !e;

o3 = T14;
T14 = T13 * T12;
T13 = b + a;
```

```
T12 = e * !f + ! e * f ;
```

13.5 Code Generation

In this chapter we discuss the code representation used in the back-end of the compiler. System utilities related to compilers, such as assemblers, linkers, archive managers, ELF inspection utilities are also discussed.

13.5.1 GNU Binutils

The GNU Binutils are a collection of binary tools. The main ones are:

1. `ld` - the GNU linker.
2. `as` - the GNU assembler.

All the tools within the GNU binutils toolset are listed below:

1. `addr2line`: converts addresses into filenames and line numbers,
2. `ar`: a utility for creating, modifying and extracting from archives,
3. `c++filt`: filter to demangle encoded C++ symbols,
4. `gold`: a new, faster, ELF only linker,
5. `gprof`: displays profiling information,
6. `nm`: lists symbols from object files,
7. `objcopy`: copys and translates object files,
8. `objdump`: displays information from object files,
9. `ranlib`: generates an index to the contents of an archive,
10. `readelf`: displays information from any ELF format object file,
11. `size`: lists the subsection sizes of an object or archive file,
12. `strings`: lists printable strings from files,
13. `strip`: discards symbols,
14. `BFD`: binary file descriptor,
15. `opcodes`: opcode generalization and indirection library.

We discuss each of them in the following sections.

13.5.1.1 GNU Binutils : addr2line and libunwind

We first show the usage of `addr2line`.

```
Usage: addr2line [option(s)] [addr(s)]
 Convert addresses into line number/file name pairs.
  -b --target=<bfdname>  Set the binary file format
  -e --exe=<executable>  Set the input file name
```

```
-i --inlines            Unwind inlined functions
-f --functions          Show function names
-C --demangle[=style]   Demangle function names
-h --help               Display this information
-v --version            Display the program's version
```

Consider the C language program:

```
#include <stdio.h>
#define UNW_LOCAL_ONLY
#include <libunwind.h>

void show_backtrace (void) {
  unw_cursor_t cursor; unw_context_t uc;
  unw_word_t ip, sp;

  unw_getcontext(&uc);
  unw_init_local(&cursor, &uc);
  while (unw_step(&cursor) > 0) {
    unw_get_reg(&cursor, UNW_REG_IP, &ip);
    unw_get_reg(&cursor, UNW_REG_SP, &sp);
    printf ("ip = %lx, sp = %lx\n", (long) ip, (long) sp);
  }
}

int function_one( int x, int y ) {
  if ( y == 0 ) { show_backtrace(); return 0; }
  return (x+2)/y;
}

int function_two( int y ) {
  return function_one( y,0 ) + 2;
}

int main( ) {
  function_two( 12 );
  return 0;
}
```

Listing 13.6 Example of using `addr2line`

We compile this program with debug as follows:

```
$gcc -ggdb -c adr_example.c -o adr_example
```

The advantage of using `addr2line` is present when debugging a crash where the stacktrace contains only function addresses. Using this program these function addresses can be translated into file name and location. By using the `libunwind` library we can debug the divide-by-zero problem. Running the executable gives us the back trace:

```
ip = 80485df, sp = bfe22490
ip = 804860f, sp = bfe224a0
ip = 8048629, sp = bfe224c0
ip = 126bb6, sp = bfe224e0
ip = 8048491, sp = bfe22560
```

On a simple executable compiled with debug symbols, running `addr2line` returns:

```
$addr2line -e adr_example -f 80485df 804860f 8048629
function_one
/home/skoranne/MYBOOK/adr_example.c:19
function_two
/home/skoranne/MYBOOK/adr_example.c:24
main
/home/skoranne/MYBOOK/adr_example.c:29
```

Another excellent usage of `addr2line` is in using the new *advice* functionality in GCC 4.5.0. See Section 3.1.4 for more details and an example use.

13.5.1.2 GNU Binutils ar : archive manager

The GNU archive manager `ar` program creates, modifies, and extracts from archives. An archive is a single file holding a collection of other files in a structure that makes it possible to retrieve the original individual files (called members of the archive). The original files contents, mode (permissions), timestamp, owner, and group are preserved in the archive, and can be restored on extraction.

```
commands:
  d            delete file(s) from the archive
  m[ab]        move file(s) in the archive
  p            print file(s) found in the archive
  q[f]         quick append file(s) to the archive
  r[ab][f][u]  replace or insert new file(s) into the archive
  t            display contents of archive
  x[o]         extract file(s) from the archive
  c            do not warn if new archive was created
  s            perform indexing
  v            be verbose
  V            display version information and exit.
```

13.5.1.3 GNU Binutils c++filt : Name de-mangler for C++

Anyone who has every programmed in C++ and used STL knows the reams of messages and long complex mangled names generated by the compiler to support everything from operator overloading to lookup resolution. Fortunately, the system also comes with a name demangler to perform the reverse translation using exactly the same rules (ABI permitting). Consider the following code fragment:

```
// example code for C++ de-mangler
#include <vector>
#include <set>
typedef std::vector<  std::set<int> > CPLX;
int function(void) {
  CPLX A;
  CPLX::const_iterator it = A.begin();
  return ( it != A.end() );
}
```

Listing 13.7 Example of using `c++filt` for demangling C++ names

Now consider a break-point associated with the symbol:

```
_ZNSaISt3setIiSt4lessIiESaIiEEEC2Ev
```

Using the `c++filt` command we can easily translate this to:

```
$c++filt _ZNSaISt3setIiSt4lessIiESaIiEEEC2Ev
std::allocator<std::set<int, std::less<int>,
   std::allocator<int> > >::allocator()
```

This informs us that the symbol is actually the memory allocator for the STL set.

13.5.1.4 GNU Binutils gprof

This utility is covered in Section 3.12.1.

13.5.1.5 GNU Binutils nm

The `nm` utility prints symbols present in object and executable files.

```
Usage: nm [option(s)] [file(s)]
 List symbols in [file(s)] (a.out by default).
 The options are:
  -a, --debug-syms
  -A, --print-file-name
  -C, --demangle[=STYLE]
      --no-demangle
  -g, --extern-only
  -l, --line-numbers

  -n, --numeric-sort
  -S, --print-size
  -V, --version
```

The utility prints the function names of referenced functions as well. Consider the `nm` utility run on an executable:

```
$nm adr_example
...
08048500 t frame_dummy
080485ce T function_one
080485f6 T function_two
         U getcontext@@GLIBC_2.1
08048614 T main
         U printf@@GLIBC_2.0
```

The nm command can be used to create a map file (a map file lists the address of functions in the binary and the corresponding function name). A map file can be passed as an input to the linker program (see Section 13.5.3). which will respect the addressing information. It can also be reordered, based on profile data (see Section 3.12.1). To create a map file using nm:

```
nm --extern-only --defined-only -v --print-file-name <object-file>
```

The produced map file has the following syntax:

```
cano_proc:08049690 T _init
cano_proc:08049cc0 T _start
cano_proc:08049cf0 T __gmon_start__
cano_proc:0804a1cb T _ZlsRSoRK22PolyRep
cano_proc:0804a97c T _ZN22PolyRep13RKSt6vectorIiSaIiEERSs
cano_proc:0804b0e4 T _ZN22PolyRep11ComputeRankEv
cano_proc:0804b12a T _ZN22PolyRep19CalculatePropertiesEv
cano_proc:0804dccc T _ZN13DP14parse_verticesEj
cano_proc:0804eb1c T _ZN13DP22parse_vertices_reducedEj
cano_proc:0804f406 T _ZN13DP12new_polytopeEv
cano_proc:0804f424 T _ZN13DP3RunEv
cano_proc:0804fe64 T main
```

13.5.1.6 GNU Binutils objcopy

The GNU objcopy utility copies the contents of an object file to another. GNU objcopy uses the GNU BFD Library to read and write the object files. It can write the destination object file in a format different from that of the source object file. The exact behavior of GNU objcopy is controlled by command-line options. Note that GNU objcopy should be able to copy a fully linked file between any two formats. However, copying a relocatable object file between any two formats may not work as expected.

```
Copies a binary file, possibly transforming it in the process
The options are:
 -I --input-target <bfdname>
 -O --output-target <bfdname>
 -p --preserve-dates
 -j --only-subsection <name>
    --add-gnu-debuglink=<file>
 -R --remove-subsection <name>
 -S --strip-all
```

A more complete set of command line options can be found by running man objcopy.

13.5.1.7 GNU Binutils objdump

GNU objdump displays information about one or more object files. The options control what particular information to display. This information is mostly useful

to system programmers who are working on the compilation tools, as opposed to programmers who just want their program to compile and work.

```
Usage: objdump <option(s)> <file(s)>
 Display information from object <file(s)>.
 At least one of the following switches must be given:
  -a, --archive-headers
  -f, --file-headers
  -p, --private-headers
  -h, --[subsection-]headers
  -x, --all-headers
  -d, --disassemble
  -D, --disassemble-all
  -S, --source
  -s, --full-contents
  -g, --debugging
  -e, --debugging-tags
  -G, --stabs
  -W[lLiaprmfFsoR] or
  -t, --syms
  -T, --dynamic-syms
  -r, --reloc
  -R, --dynamic-reloc
  @<file>
  -v, --version
  -i, --info
  -H, --help
```

Running GNU `objdump` on an existing archive gives the (i) permission, (ii) file size, (iii) file modification time stamp, and (iv) name of object file for all objects in the archive. Example:

```
In archive /home/skoranne/OSS/lib/libelf.a:

begin.o:     file format elf32-i386
rw-rw-r-- 500/500  18308 Jun 12 11:15 2010 begin.o

cntl.o:     file format elf32-i386
rw-rw-r-- 500/500  10984 Jun 12 11:15 2010 cntl.o
```

13.5.1.8 GNU Binutils ranlib

GNU `ranlib` generates an index to the contents of an archive and stores it in the archive. The index lists each symbol defined by a member of an archive that is a relocatable object file. You may use `nm -s` or `nm --print-armap` to list this index. An archive with such an index speeds up linking to the library and allows routines in the library to call each other without regard to their placement in the archive. Usage:

```
  -a,   archive header information
  -f,   display file header
  -p,   display format specific object header
  -h,   display subsection headers
  -x,   display all headers
  -d,   show assembler section
  -S,   show source code as well
  -s,   show full contents of subsections.
```

A more complete list of command line options can be found by running `man ranlib`.

13.5.1.9 GNU Binutils readelf

The `readelf` command displays information about one or more ELF format object files. The options control what particular information to display. For example, if we run `readelf adr_example` we get:

```
$readelf -h adr_example
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 03 00 00 00 00 00 00 00 00
  Class:                             ELF32
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - Linux
  ABI Version:                       0
  Type:                              EXEC (Executable file)
  Machine:                           Intel 80386
  Version:                           0x1
  Entry point address:               0x8048470
  Start of program headers:          52 (bytes into file)
  Start of subsection headers:          5640 (bytes into file)
  Flags:                             0x0
  Size of this header:               52 (bytes)
  Size of program headers:           32 (bytes)
  Number of program headers:         8
  Size of subsection headers:           40 (bytes)
  Number of subsection headers:         38
  Subsection header string table index: 35
```

The command line options for `readelf` are:

```
Usage: readelf <option(s)> elf-file(s)
 Display information about the contents of ELF format files
 Options are:
  -a --all               Equivalent to: -h -l -S -s -r -d -V -A -I
  -h --file-header       Display the ELF file header
  -l --program-headers   Display the program headers
     --segments          An alias for --program-headers
  -S --subsection-headers   Display the subsections' header
     --subsections          An alias for --subsection-headers
  -e --headers           Equivalent to: -h -l -S
```

A more complete list of command line options can be found by running `man readelf`.

13.5.1.10 GNU Binutils size

The GNU `size` utility lists the subsection sizes—and the total size—for each of the object or archive files objfile in its argument list. By default, one line of output is generated for each object file or each module in an archive.

```
Usage: size [option(s)] [file(s)]
Displays sizes of subsections
  -t --totals           Display the total sizes
     --common           Display total size
     --target=<bfdname> Set binary file format
```

For example if we run `size adr_example` we get:

```
$size -d adr_example
   text    data     bss     dec     hex filename
   1616     276       8    1900     76c adr_example
```

13.5.1.11 GNU Binutils strings

For each file given, GNU strings prints the printable character sequences that are at least 4 characters long (or the number given with the options below) and are followed by an unprintable character. By default, it only prints the strings from the initialized and loaded subsections of object files; for other types of files, it prints the strings from the whole file.

```
 The options are:
  -a - --all    Scan the entire file
  -f            Print name of file as well
  -n --bytes=[n] change minimum len to 4
  -<number> least [number] characters (default 4).
```

A more complete list of all command line options can be generated by running `man strings`. Consider the following program fragment:

```
const char RCS_ID[] = "IDversion1.234";
int function( int x ) { return x+1; }
```

Listing 13.8 Example of using the `strings` program

This program when compiled into object file still contains the string containing the version string. Using the `strings` command we can find the version string embedded inside it.

```
$strings rcs.o
$ID version 1.234$
```

13.5.1.12 GNU Binutils strip

GNU `strip` discards all symbols from object files objfile. The list of object files may include archives. At least one object file must be given. GNU `strip` modifies the files named in its argument, rather than writing modified copies under different names. It is useful in reducing program size before production, which makes the program load faster. Usage:

```
Usage: strip <option(s)> in-file(s)
 Removes symbols and subsections from files
 The options are:
  -I --input-target=<bfdname>
  -O --output-target=<bfdname>
  -p --preserve-dates
  -R --remove-subsection=<name>
  -s --strip-all
  -K --keep-symbol=<name>
  -o <file>
```

13.5.2 GNU Binutils libelf and elfutils

GNU libelf lets you read, modify or create ELF files in an architecture-independent way. The library takes care of size and endian issues, e.g. you can process a file for SPARC processors on an Intel-based system.

```
/*
 * Example of libelf
 * (C) Sandeep Koranne, 2010
 */
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include "libelf.h"

int main( int argc, char* argv[] ) {
  Elf *elf_ptr;
  Elf_Kind elf_k;
  int fd;
  char *c;
  if( argc != 2 ) {
    fprintf( stderr, "usage: poke_elf <filename>" );
    exit(1);
  }
  if( elf_version( EV_CURRENT ) == EV_NONE ) {
    fprintf( stderr, "ELF library init failed");
    exit(1);
  }

  fd = open( argv[1], O_RDONLY, 0 );
  if( fd < 0 ) {
    fprintf( stderr, "Unable to open file %s", argv[1] );
    exit(1);
  }

  elf_ptr = elf_begin( fd, ELF_C_READ, NULL );
  if( elf_ptr == NULL ) {
```

```
    fprintf( stderr, "elf_begin() failed : %s", elf_errmsg(-1) );
    exit(1);
  }

  elf_k = elf_kind( elf_ptr );

  switch( elf_k ) {
  case ELF_K_ELF: fprintf( stdout, "ELF object" ); break;
  case ELF_K_AR : fprintf( stdout, "ar(1) archive" ); break;
  case ELF_K_NONE:fprintf( stdout, "data" ); break;
  default: fprintf( stdout, "_NONE_" ); break;
  }
  elf_end( elf_ptr );
  close( fd );
  exit( 0 );
}
```

Listing 13.9 Example of using `libelf` library

We can compile this file as:

```
$gcc elf_poke.c -o elf_poke -lelf
```

And now we have an ELF detection tool which can distinguish between ELF objects and 'ar' archives. We run the executable on itself:

```
$./elf_poke elf_poke
ELF object[
$./elf_poke libelf.a
ar(1) archive
```

As shown in Figure 13.2 ever ELF has the following data:

1. ELF executable header: describes the *class* (whether the file is 32-bit or 64-bit), *type* (whether the file is relocatable, shared or executable), and the *endianess* of the file,
2. Optional ELF program header table (PHDR)
3. Subsection Data and the subsection header table are used for relocatable data.

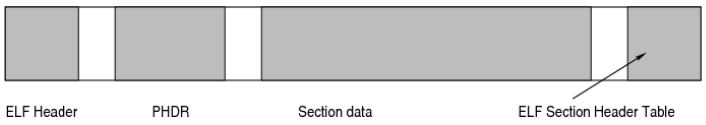


Fig. 13.2 Layout of an ELF file

The `libelf` library contains many examples which show how to analyze the subsections and headers present inside the ELF file. Using this library interesting and useful programs can be built for program maintenance and analysis.

13.5.3 GNU Binutils ld

A compiler translates program code (given as human readable text) into into a machine specific but still readable text called assembler code. Assembly code is a readable form of the machine code (CPU instructions) which the computer can execute directly. A linker converts object files into executables and shared libraries. GNU linker (or GNU ld) is the GNU Project's implementation of the Unix command ld. GNU `ld` runs the linker, which creates an executable file (or a library) from object files. A linker script may be passed to GNU `ld` to exercise greater control over the linking process. GNU `ld` combines a number of object and archive files, relocates their data and ties up symbol references. Usually the last step in compiling a program is to run `ld`. If the linker is being invoked indirectly, via a driver such as `gcc` then all the linker command line options should be prefixed by -Wl, as shown below:

```
$gcc -Wl,--start-group foo.o bar.o -Wl,--end-group
```

GNU `ld` command supports a number of options which control its behavior. The common options are listed below:

```
‘-E’
‘--export-dynamic’
‘--no-export-dynamic’

‘-L SEARCHDIR’
‘--library-path=SEARCHDIR’
     Add SEARCHDIR to list of paths that ‘ld’ will search for

‘-M’
‘--print-map’

‘-o OUTPUT’
‘--output=OUTPUT’

‘-r’
‘--relocatable’
     Generate relocatable output

‘-Bdynamic’
‘-dy’
‘-call_shared’
     Link against dynamic libraries.

‘-Bstatic’
‘-dn’
‘-non_shared’
‘-static’
     Do not link against shared libraries.

‘--cref’
     Output a cross reference table.

‘--demangle[=STYLE]’
```

```
'--no-demangle'
```

For a complete list of command line options for `ld`, run `man ld`, or `info ld`.

13.5.3.1 Linker script and map files

When passing arguments to the linker, the GNU gcc commandline `-Wl` as follows:

```
$gcc file.o -o file -Wl, -M
```

will generate the linker map for the file. The format of the linker map is discussed below, an example is:

```
 .group          0x0000000000000000        0x8 cano_proc.o
 _ZN9__gnu_cxx13new_allocatorISt6vectorIiSaIiEEEC5ERKS4_
                 0x0000000000000000        0x8 cano_proc.o
 .group          0x0000000000000000        0x8 cano_proc.o
 .group          0x0000000000000000        0x8 cano_proc.o
 .group          0x0000000000000000        0x8 cano_proc.o
 .text._ZNSt12__niter_baseIPSt6vectorI7CBTupleSaIS1_EELb0EE3__bES4_
                 0x0000000000000000        0xd main.o
.text._ZNSt3setIiSt4lessIiESaIiEE6insertERKi
                 0x0000000008050bb4       0x5f cano_proc.o
                 0x0000000008050bb4     std::set<int, std::less<int>
 *fill*          0x0000000008050c13        0x1 90909090
...
/DISCARD/
 *(.note.GNU-stack)
 *(.gnu_debuglink)
OUTPUT(cano_proc elf32-i386)
```

The `.group` and `.text` sections contain the instruction to the loader to place code and data segments into appropriate sections of the memory. This arrangement of code blocks from disk to memory can be managed and optimized (for cache line performance, see Section 3.12.1.1 for more details), using a *linker script*. The linker will ensure that each output subsection has the required alignment, by increasing the location counter if necessary. In this example, the specified addresses for the '.text' and '.data' subsections will probably satisfy any alignment constraints, but the linker may have to create a small gap between the '.data' and '.bss' subsections. As with all optimizations, linker scripts should be used when the correctness of the program has been verified, and especially when using dynamic loaded shared libraries, linker map scripts should be carefully checked.

13.5.3.2 Linker map cross reference table

The cross reference table is generated using the `--cref` commandline switch to the linker. It has the form of 'function', 'object file', e.g.:

```
BFS::Run()                          poly_utils.o
BFS::end_visit(unsigned int)        poly_utils.o
BFS::expand(unsigned int)           poly_utils.o
BFS::visit(unsigned int)            poly_utils.o
```

When passing arguments to the linker, the GNU gcc commandline `-Wl` as follows:

```
$gcc file.o -o file -Wl, --cref
```

will generate the cross reference table for the file.

13.5.4 BFD: Binary File Descriptor Library

The BFD, or Binary File Descriptor library, is the GNU Project's main mechanism for the portable manipulation of object files in a variety of formats. BFD works by presenting a common abstract view of object files. An object file has a "header" with descriptive info; a variable number of "subsections" that each have a name, some attributes, and a block of data; a symbol table; relocation entries; and so forth. To use the library, include 'bfd.h' and link with 'libbfd.a'. BFD provides a common interface to the parts of an object file for a calling application. When an application successfully opens a target file (object or archive), a pointer to an internal structure is returned. This pointer points to a structure called 'bfd', described in 'bfd.h'. Our convention is to call this pointer a BFD, and instances of it within code abfd. All operations on the target object file are applied as methods to the BFD.

```
#include "bfd.h"
unsigned int number_of_subsections (abfd)
bfd *abfd;
{
  return bfd_count_subsections (abfd);
}
```

The abstraction used within BFD is that an object file has:

1. a header,
2. a number of subsections containing raw data,
3. a set of relocations,
4. some symbol information.

Also, BFDs opened for archives have the additional attribute of an index and contain subordinate BFDs. The design and use of BFD can be organized as follows:

- BFD Front end :
  1. Initialization,
  2. Subsections,
  3. Symbols,
  4. Archives,
  5. File formats,

6. Relocations,
7. Core files,
8. Targets,
9. Architectures,
10. Opening and closing BFDs,
11. File caching,
12. Linker functions,
13. Hash tables.

- BFD Back end:

1. a.out backends,
2. coff backends,
3. elf backends.

13.5.5 GNU lightning

Dynamic code generation is the generation of machine code at runtime. It is typically used to strip a layer of interpretation by allowing compilation to occur at runtime. One of the most well-known applications of dynamic code generation is perhaps that of interpreters that compile source code to an intermediate bytecode form, which is then recompiled to machine code at run-time. For performance, gnu lightning emits machine code without first creating intermediate data structures such as RTL representations traditionally used by optimizing compilers.

13.5.5.1 GNU lightning instruction set

GNU lightning's instruction set was designed by deriving instructions that closely match those of most existing RISC architectures. The library supports a full range of integer types: operands can be 1, 2 or 4 bytes long (64-bit architectures might support 8 bytes long operands), either signed or unsigned. The types are listed in the following table together with the C types they represent: There are at least seven integer registers, of which six are general-purpose, while the last is used to contain the frame pointer (FP). The frame pointer can be used to allocate and access local variables on the stack, using the allocai instruction. A partial list of instruction is given below in Table 13.2.

13.5.5.2 Instructions in GNU lightning

The instruction set of GNU lightning is modeled after a generic RISC machine. The list of instructions categorized by their function is shown in Table 13.2.

The following example from GNU lightning is shown:

Table 13.1 Data types supported by GNU lightning

Symbol	C Data Type
c	signed char
uc	unsigned char
s	short
us	unsigned short
i	int
ui	unsigned int
l	long
ul	unsigned long
f	float
d	double
p	void *

```
#include <stdio.h>
#include "lightning.h"
static jit_insn codeBuffer[1024];
typedef int (*pifi)(int);
int main() {
    pifi  incr = (pifi) (jit_set_ip(codeBuffer).iptr);
    int   in;
    jit_leaf(1);                      /*      leaf  1             */
    in = jit_arg_i();                 /* in = arg_i               */
    jit_getarg_i(JIT_R0, in);         /*      getarg_i R0         */
    jit_addi_i(JIT_RET, JIT_R0, 1);   /*      addi_i   RET, R0, 1 */
    jit_ret();                        /*      ret                 */

    jit_flush_code(codeBuffer, jit_get_ip().ptr);

    /* call the generated code, passing 5 as an argument */
    printf("%d + 1 = %d\n", 5, incr(5));
    return 0;
}
```

Listing 13.10 Example of using GNU lightning

GNU lightning generates the following code on SPARC:

```
save %sp, -96, %sp
mov  %i0, %l0      retl
add  %l0, 1,  %i0  add %o0, 1, %o0
ret
restore
```

Table 13.2 Instruction in GNU lightning

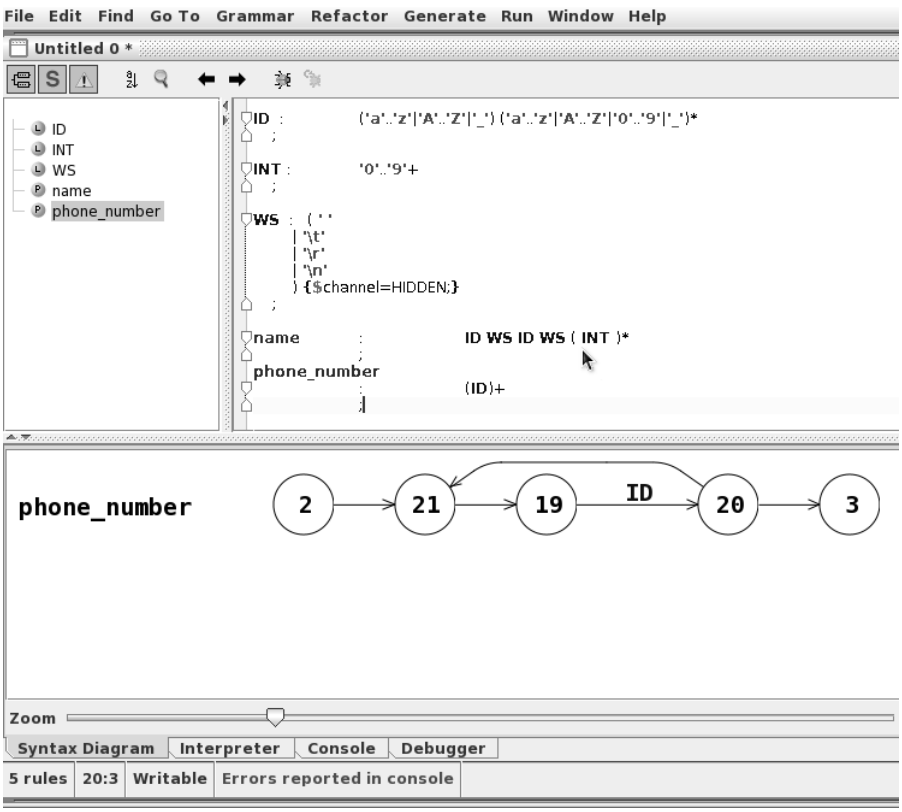
ALU Instructions	
`add[r,i]`	`O1 = O2 + O3`
`sub[r,i]`	`O1 = O2 - O3`
`mul[r,i]`	`O1 = O2 * O3`
`div[r,i]`	`O1 = O2 / O3`
`mod[r,i]`	`O1 = O2 % O3`
`and``[r,i]`	`O1 = O2 & O3`
`or``[r,i]`	`O1 = O2 | O3`
`xor``[r,i]`	`O1 = O2 ^ O3`
Compare instructions	
`lt[r,i]`	`O2 < O3`
`le[r,i]`	`O2 <= O3`
`gt[r,i]`	`O2 > O3`
`ge[r,i]`	`O2 >= O3`
`eq[r,i]`	`O2 == O3`
`ne[r,i]`	`O2 != O3`
Load and Store Instructions	
`ld[r,i]`	`O1 = *O2`
`ldx[r,i]`	`O1 = *(O2+O3)`
`st[r,i]`	`*O1 = O2`
`stx[r,i]`	`*(O1+O2) = O3`
Branch Instructions	
`blt[r,i]`	if (O2 $<$ O3) goto O1
`ble[r,i]`	if (O2 $\leq$ O3) goto O1
`bgt[r,i]`	if (O2 $>$ O3) goto O1
Jump and Return Instructions	
`call[r,i]`	function call to O1
`finish`	function call to O1
`finishr`	function call to a register
`jmp[r,i]`	unconditional jump to O1
`ret`	return from subroutine
`retval`	move return value

13.5.6 ANTLR

Before we move on to a compiler optimization and representation framework (LLVM) it is only apt to mention ANTLR, which is a complete suite of compiler construction tools. An example of the ANTRL screen and DFA definition is shown in Figure 13.3.



(a) Introductory screen



(b) DFA of phone number grammar in ANTLR

Fig. 13.3 ANTLRWorks introductory screen

13.6 LLVM: Low Level Virtual Machine

LLVM (Low Level Virtual Machine), is a compiler framework designed to support transparent, lifelong program analysis and transformation for arbitrary programs, by providing high-level information to compiler transformations at compile-time, link-time, run-time, and in idle time between runs. LLVM defines a common, low-level code representation in Static Single Assignment (SSA) form, with several novel features: a simple, language-independent type-system that exposes the primitives commonly used to implement high-level language features; an instruction for typed address arithmetic; and a simple mechanism that can be used to implement the exception handling features of high-level languages (and setjmp/longjmp in C) uniformly and efficiently. The main projects under the LLVM are (from http://www.llvm.org):

- LLVM Core: the LLVM core provides source and target independent optimization based on SSA, alongwith code generation for modern architectures. The LLVM intermediate representation is well specified and documented,
- Clang: is an LLVM native, C/C++/Objective-C compiler,
- llvm-gcc and dragonegg: integration of LLVM with GCC frontend,
- LLDB: using LLVM and Clang to produce native debugger.

LLVM core and LLVM IR (intermediate representation) are useful in compiler construction, as they provide a generic optimization and instruction scheduling, generation system. Using the front-end tools (Flex and Bison) alongwith LLVM a prototypical compiler can be constructed rapidly, and with ease.

13.6.1 LLVM Core and LLVM IR

Above we saw the GNU lightning (Subsection 13.5.5) and its associated RISC instruction model. In this section we present the LLVM instructions and their intermediate representations.

13.6.1.1 LLVM Intermediate Representation

The LLVM IR can be thought of as the assembly language, or mnemonic instruction set of the low-level virtual machine (LLVM). LLVM is a Single Static Assignment (SSA) representation that provides type safety, flexibility and sufficient breadth to implement all the high level language constructs. LLVM code representation was designed to be used in the following scenarios:

1. in-memory compiler IR,
2. on-disk instruction bit-code, and
3. human readable assembly language.

The use of LLVM instructions are shown below:

```
%result = mul i32 %X, 8
```

Without even going into specifics, it is clear that the above multiplies the content of variable %X with 8, and places the result in %result. LLVM's optimization can do *strength reduction* to convert the above multiply by 8 to a shift left by 3 operation to get:

```
%result = shl i32 %X, i8 3
```

Before we present more examples of LLVM textual IR, and the instructions, we should note that:

- LLVM comments: are delimited by ';' character
- unnamed temporaries: are created when lval is not present, and
- unnamed temporaries: are numbered sequentially.

13.6.1.2 LLVM Program High Level Structure

LLVM programs are composed of *Modules*, each of which corresponds to a translation unit (mostly a single file) of the input program. Each module consists of:

1. functions,
2. global variables,
3. symbol table entries.

Alongwith the module composition, the Linkage types, and calling conventions for the function calls are important considerations when designing an LLVM based IR. LLVM also allows for type aliasing, for example, we can define:

```
%point_type = type { %point_type*, i32 }
```

Functions defined using LLVM have (i) linkage types, (ii) visibility types, (iii) calling convention, (iv) return type, (v) parameter attribute for return type, (vi) function name, (vii) argument list (possibly empty), (viii) optional subsection, (ix) optional alignment, (x) optional garbage collector name, (xi) opening curly brace, (xii) list of basic blocks, and (xiii) closing curly braces. For example:

```
extern int fibo(int n);
=>
define i32 @fibo(i32 %n) nounwind readnone {

static int fibo(int n);
=>
define internal fastcc i32
           @fibo(i32 %n) nounwind readnone {
```

The function attributes and linkage vary depending upon the data present in the translation unit. The actual code of the function:

```
static int fibo( int n ) {
  int i,f0=0,f1=1,f;
  for(i=0; i < n; ++i) {
    f = f0 + f1;
    f0 = f1;
    f1 = f;
  }
  return f;
}
```

is assembled by LLVM into:

```
define internal fastcc i32 @fibo(i32 %n)
   nounwind readnone {
entry:
  %0 = icmp sgt i32 %n, 0                ; <i1> [#uses=1]
  br i1 %0, label %"4", label %"5"

"4":                                     ; preds = %"4", %entry
  %1 = phi i32 [ %4, %"4" ], [ 1, %entry ; <i32> [#uses=2]
  %2 = phi i32 [ %1, %"4" ], [ 0, %entry ; <i32> [#uses=1]
  %3 = phi i32 [ %5, %"4" ], [ 0, %entry ; <i32> [#uses=1]
  %4 = add nsw i32 %1, %2                ; <i32> [#uses=2]
  %5 = add nsw i32 %3, 1                 ; <i32> [#uses=2]
  %exitcond = icmp eq i32 %5, %n         ; <i1> [#uses=1]
  br i1 %exitcond, label %"5", label %"4"

"5":                                     ; preds = %"4", %entry
  %.lcssa = phi i32 [ undef, %entry ],
   [ %4, %"4" ] ; <i32> [#uses=1]
  ret i32 %.lcssa
}
```

Function attributes are an important part of the optimization performed by LLVM. The following are some of the function attributes that can be specified per function (their use is self explanatory):

- alignstack
- alwaysinline
- noinline
- optsize
- noreturn
- nounwind
- readnone
- readonly

13.6.1.3 LLVM Instruction Summary: Terminators

The following table presents the terminator instructions in LLVM. Every basic block must end with one of these instruction: The computation instructions in LLVM are standard RISC style instructions; a short summary is presented in the following table.

Table 13.3 LLVM Terminator instructions

ret	ret <void> or ret <type> <value>
br	br <label> or br i1 <cond>, label <iftrue>, label <iffalse>
switch	switch <intty> <value>, label <defaultdest> [..]
indirectbr	indirect branch
invoke	transfer control to specified function
unwind	unwind the stack, transfer control to first callee,
unreachable	optimizer hint

13.6.1.4 LLVM Instruction Summary: Computation

Table 13.4 LLVM Instructions for computation

add	= add <ty> <op1> <op2>
sub	= sub <ty> <op1> <op2>
mul	= mul <ty> <op1> <op2>
div	= div <ty> <op1> <op2>
rem	= rem <ty> <op1> <op2>
shl	= shl <ty> <op1> <op2>
lshr	= lshr <ty> <op1> <op2>
ashr	= ashr <ty> <op1> <op2>
and	= and <ty> <op1> <op2>
or	= or <ty> <op1> <op2>
xor	= xor <ty> <op1> <op2>

There are some other special instructions:

- Vector *extractelement*,
- Vector *insertelement*,
- Vector *shufflevector*,
- Memory *alloca*,
- *load*,
- *store*,
- *ptrtoint*,
- *inttoptr*,
- *icmp*,
- *fcmp*,
- *phi*: implement the ϕ node in SSA graph,
- *call*: function call instruction.

LLVM also supports a number of intrinsic functions, a representative set is shown below:

- llvm.va_start
- llvm.gcroot
- llvm.returnaddress
- llvm.stacksave
- llvm.prefetch
- llvm.readycyclecounter
- llvm.memcpy
- llvm.sqrt.*
- llvm.bswap.*
- llvm.ctpop.* :
- llvm.cttz.* :
- llvm.memory.barrier llvm.atomic.load.add.* and sub

13.6.2 LLVM dragonegg

LLVM dragonegg is a plugin to GCC, thus it requires a fairly recent GCC version. GCC 4.5.0 works with dragonegg 2.7, and this version is used in the following examples. Consider the following C language program:

```
int function_one( int x ) {
  int i;
  int sum;
  for( i = 0; i < x; ++i ) sum += i;
  return sum;
}
```

Compiling this program with GCC using the -S option produces the assembly code (this assembly code is for x86 architecture):

```
        .file   "sum.c"
        .text
.globl function_one
        .type   function_one, @function
function_one:
        pushl   %ebp
        movl    %esp, %ebp
        subl    $16, %esp
        movl    $0, -8(%ebp)
        jmp     .L2
.L3:
  movl    -8(%ebp), %eax
  addl    %eax, -4(%ebp)
  addl    $1, -8(%ebp)
.L2:
  movl    -8(%ebp), %eax
  cmpl    8(%ebp), %eax
  jl      .L3
  movl    -4(%ebp), %eax
  leave
  ret
```

```
  .size   function_one, .-function_one
  .ident  "GCC: (GNU) 4.4.2 20091027 (Red Hat 4.4.2-7)"
  .subsection        .note.GNU-stack,"",@progbits
```

We can use the `dragonegg` plugin to use the LLVM intermediate representation, optimization and scheduler. Running:

```
$gcc -fplugin=dragonegg.so -O1 -S sum.c
```

produces:

```
        .file   "sum.c"
# Start of file scope inline assembly
        .ident  "GCC: (GNU) 4.5.0 LLVM: exported"
# End of file scope inline assembly
        .text
        .globl  function_one
        .align  16, 0x90
        .type   function_one,@function
function_one:
        pushl   %ebp
        movl    %esp, %ebp
        pushl   %edi
        pushl   %esi
        subl    $28, %esp
        movl    8(%ebp), %eax
        movl    %eax, -12(%ebp)
        movl    -12(%ebp), %eax
        movl    %eax, -20(%ebp)
        movl    $0, %eax
        movl    %eax, -24(%ebp)
        jmp     .LBB1_3
.LBB1_2:
        movl    -28(%ebp), %eax
        movl    %eax, %ecx
        movl    -32(%ebp), %edx
        addl    %edx, %ecx
        movl    %edx, %esi
        addl    $1, %esi
        movl    %ecx, -36(%ebp)
        movl    %esi, -24(%ebp)
.LBB1_3:
        movl    -24(%ebp), %eax
        movl    %eax, %ecx
        movl    -36(%ebp), %edx
        movl    %edx, %esi
        movl    -20(%ebp), %edi
        cmpl    %edi, %ecx
        movl    %ecx, -32(%ebp)
        movl    %esi, -28(%ebp)
        jl      .LBB1_2
        movl    -28(%ebp), %eax
        movl    %eax, -16(%ebp)
        movl    -16(%ebp), %eax
        addl    $28, %esp
```

```
        popl    %esi
        popl    %edi
        popl    %ebp
        ret
        .size   function_one, .-function_one

        .subsection .note.GNU-stack,"",@progbits
```

To produce the LLVM IR we can add

`-fplugin-arg-dragonegg-emit-ir`

to produce:

```
; ModuleID = 'sum.c'
target triple = "i386-pc-linux-gnu"
module asm "\09.ident\09\22GCC: (GNU) 4.5.0 LLVM: exported\22"
%int = type i32

define i32 @function_one(i32 %x) nounwind {
entry:
  %x_addr = alloca i32                        ]
  %memtmp = alloca i32                        ]
  %"alloca point" = bitcast i32 0 to i32
  store i32 %x, i32* %x_addr
  %0 = load i32* %x_addr, align 32
  %"ssa point" = bitcast i32 0 to i32
  br label %"2"

"2":
  br label %"4"

"3":
  %1 = add nsw i32 %4, %3
  %2 = add nsw i32 %3, 1
  br label %"4"

"4":
  %3 = phi i32 [ %2, %"3" ], [ 0, %"2" ]
  %4 = phi i32 [ %1, %"3" ], [ undef, %"2" ]
  %5 = icmp slt i32 %3, %0
  br i1 %5, label %"3", label %"5"

"5":
  store i32 %4, i32* %memtmp, align 1
  br label %return

return:
  %retval = load i32* %memtmp
  ret i32 %retval
}
```

Thus, using the plugin we can quickly experiment with LLVM, while maintaining the front-end capabilities of GCC. It is indeed possible to compile some parts of an

application with LLVM based tools, while compiling the remaining with GCC. In the next section we describe the LLVM compiler and optimization system.

13.6.3 LLVM System

The LLVM system for the end-user (not the developer) comprises of a number of programs and utilities which manipulate, optimize and analyze LLVM '.ll' and '.bc' files. The utilities are:

1. `lli`: LLVM interpreter and dynamic compiler,
2. `llc`: LLVM system compiler and optimizer,
3. `llvm-ar`: LLVM archiver, archives several bit-code files into a single archive. It can optionally compress the members to save space, and it generates a symbol table for efficient lookup during linking,
4. `llvm-as`: LLVM assembler (.ll to .bc),
5. `llvm-bcanalyzer`: LLVM bit-code analyzer,
6. `llvmc`: LLVM compiler driver,
7. `llvm-dis`: LLVM disassembler. It takes the LLVM bit-code file and converts it to LLVM assembly,
8. `llvm-extract`: this command takes the name of a function and extracts that function's code from the bit-code file,
9. `llvm-ld`: LLVM linker, takes a number of LLVM bit-code files and links them together into a single LLVM object. It can also produce native code executables,
10. `llvm-nm`: GNU nm equivalent for LLVM, reports names of symbols from the LLVM bit-code file,
11. `llvm-prof`: LLVM profiler, reads in a `llvmprof.out` file, a bit-code program file, and produces reports, which can be used to deduce program's hotspots,
12. `llvm-ranlib`: adds or updates the symbol table in an LLVM archive file,
13. `llvm-stub`:

We discuss some of the important utilities below.

13.6.3.1 LLVM interpreter and dynamic compiler: lli

LLVM `lli` can directly execute programs in LLVM bitcode ('.bc') format. It uses JIT (just in time) compilation on the bit-code for JIT supported architectures, else it uses an interpreter. LLVM `lli` has a number of command-line options which control its behavior, including floating-point operation, code model (to choose from (i) small, (ii) kernel, (iii) medium, and (iv) large). It can also be used as a JIT compiler, and for this it accepts the `-march` and `-mcpu` command-line options.

13.6.3.2 LLVM System Compiler

Once we have generated the bit-code from the LLVM assembler we can experiment with optimization and instruction scheduling, and other advanced compiler optimizations. One of the design goals of LLVM is that such optimization will happen concurrently with the life of the executable. In the context of mobile applications, this is not as far-fetched as originally thought.

We use `llc` to recompile the bit-code of our Fibonacci example:

```
$llc -O3  fibo.bc -o fibo.opt.bc -stats \
                  -tailcallopt  -time-passes
```

We generate statistics about the optimization passes as well as the number of machine instructions generated, instructions scheduled.

13.6.3.3 Statistics for fibo.bc

```
23 asm-printer      -  No. machine instrs printed
  1 branchfolding    - No. branches optimized
  1 branchfolding    - No. dead blocks removed
  1 code-placement   - No. intra loop branches moved
  1 code-placement   - No. loop header aligned
  7 dagcombine       - No. dag nodes combined
102 liveintervals    - No. original intervals
  1 loop-reduce      - No. IV uses strength reduced
  1 loop-reduce      - No. PHIs inserted
  1 loop-reduce      - No. loop terminating conds optimized
  2 loopsimplify     - No. pre-header or exit blocks inserted
  1 machine-sink     - No. machine instructions sunk
  4 phielim          - No. atomic phis lowered
  2 regalloc         - No. iterations performed
 22 regcoalescing    - No. identity moves eliminated
  1 regcoalescing    - No. instructions re-materialized
 20 regcoalescing    - No. interval joins performed
  2 regcoalescing    - No. valno def marked dead
  1 scalar-evolution - No. loops with predictable loop counts
  1 twoaddrinstr     - No. instructions commuted to coalesce
  2 twoaddrinstr     - No. two-address instructions
  1 x86-codegen      - No. floating point instructions
```

13.6.3.4 Instruction scheduling

```
  --- Name ---
  DAG Legalization
  Type Legalization
  Instruction Scheduling
  Instruction Creation
  Vector Legalization
  Instruction Selection
```

```
DAG Combining 1
DAG Combining after legalize types
DAG Combining 2
Instruction Scheduling Cleanup
TOTAL
```

13.6.3.5 Compiler optimization passes

1. X86 DAG → DAG Instruction Selection
2. Live Variable Analysis
3. Simple Register Coalescing
4. Live Interval Analysis
5. Linear Scan Register Allocator
6. Loop Strength Reduction
7. X86 AT&T-Style Assembly Printer
8. Induction Variable Users
9. Optimize for code generation
10. Machine Function Analysis
11. Module Verifier
12. Canonicalize natural loops
13. Two-Address instruction pass
14. Virtual Register Map
15. Control Flow Optimizer
16. Eliminate PHI nodes for register allocation
17. Dominator Tree Construction
18. Prolog/Epilog Insertion & Frame Finalization
19. MachineDominator Tree Construction
20. Dominance Frontier Construction
21. Natural Loop Information
22. MachineDominator Tree Construction
23. Machine Natural Loop Construction
24. Remove unreachable machine basic blocks
25. Remove unreachable blocks from the CFG
26. Machine code sinking
27. Machine Natural Loop Construction
28. Exception handling preparation
29. Machine Natural Loop Construction
30. Scalar Evolution Analysis
31. X86 FP Stackifier
32. Code Placement Optimizater
33. Label Folder
34. Machine Instruction LICM
35. Stack Slot Coloring
36. Subregister lowering instruction pass
37. X86 FP_REG_KILL inserter

38. Live Stack Slot Analysis
39. X86 Maximal Stack Alignment Calculator
40. Analyze Machine Code For Garbage Collection
41. Preliminary module verification
42. Insert stack protectors
43. Lower Garbage Collection Instructions
44. Delete Garbage Collector Information

The list of compiler optimization can be better understood by referring to an advanced compiler optimization book such as Muchnik or Kennedy.

13.6.4 Using Clang

Using LLVM Clang, we can dispense with the plugin approach for compiling programs. Again consider the Fibonacci C program, but this time we compile it with clang:

```
$clang -O1 -S -c -emit-llvm -o fibo.ll fibo.c
```

The produced LLVM IR in textual form is similar to the one produced by dragonegg plugin, and it can be assembled using the LLVM assembler, `llvm-as`,

```
$llvm-as fibo.ll
```

This assembles the textual representation into the bit-code we alluded to earlier in this subsection. This bit-code can be inspected with the help of `llvm-bcanalyzer`, we present a portion of the 'fibo.bc' below:

```
Summary of fibo.bc:
         Total size: 6432b/804.00B/201W
        Stream type: LLVM IR
  # Toplevel Blocks: 1

Per-block Summary:
  Block ID #0 (BLOCKINFO_BLOCK):
      Num Instances: 1
         Total Size: 637b/79.62B/19W
          % of file: 9.903607e+00
      Num SubBlocks: 0
        Num Abbrevs: 0
        Num Records: 0

  Block ID #8 (MODULE_BLOCK):
      Num Instances: 1
         Total Size: 2270b/283.75B/70W
          % of file: 3.529229e+01
      Num SubBlocks: 7
        Num Abbrevs: 1
        Num Records: 6
      % Abbrev Recs: 1.666667e+01
```

```
        Record Histogram:
                   Count     # Bits    % Abv  Record Kind
                       3        207           FUNCTION
                       1         17  100.00   GLOBALVAR
                       1       1557           DATALAYOUT
                       1        303           TRIPLE

  Block ID #9 (PARAMATTR_BLOCK):
      Num Instances: 1
```

Once the bit-code has been generated, it can be analyzed and also optimized. For this we use the LLVM System Compiler which implements the compiler optimizations (see 13.6.3.5). The LLVM system compiler is run using `llc` command, and it has a number of useful command line options:

```
-O=<char>
 -asm-verbose
 -load=<pluginfilename>
 -march -mcpu=<cpu-name>
 -o=<filename>
 -realign-stack
 -stack-protector-buffer-size=<uint>
 -stats
 -tailcallopt
 -time-passes
 -unwind-tables
 -verify-dom-info
 -version
 -x86-asm-syntax
   =att
   =intel
```

13.7 Conclusion

Domain specific languages have re-invigorated the need for writing lexical analyzers, parsers, optimizers and instruction analysis engines. From reading configuration files, to writing domanin specific optimized languages for applications, compiler writing remains an important and integral part of scientific computing and engineering. In this chapter we have described several open-source tools for compiler writers. These include lexical analysis generators, macro processing, perfect hash generators, parser generator, and several compiler frameworks. The most important compiler framework discussed is LLVM (low level virtual machine) which provides a complete infrastructure for not only domain specific language development, but also optimization algorithms for existing languages such as C, C++ and Java.

Part IV
Engineering and Mathematical Software